



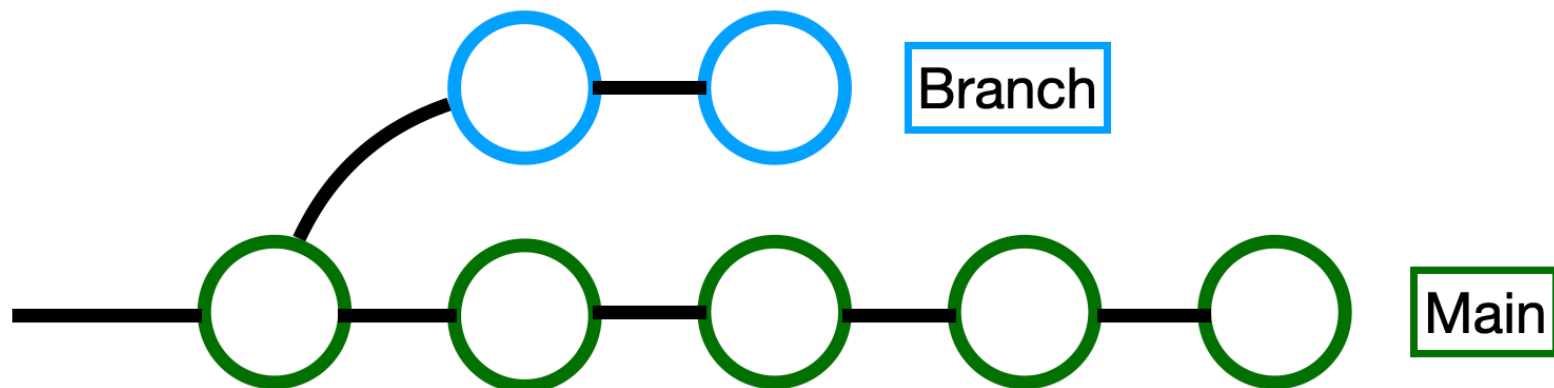
Gestão da Configuração e Versionamento

CÁSSIO CAPUCHO PEÇANHA

○ Fim da Linha Única

- Se 5 alunos da equipe programarem direto na main, a chance de um quebrar o código do outro ou enviar algo inacabado para o professor é de 100%.
- A main é sagrada.

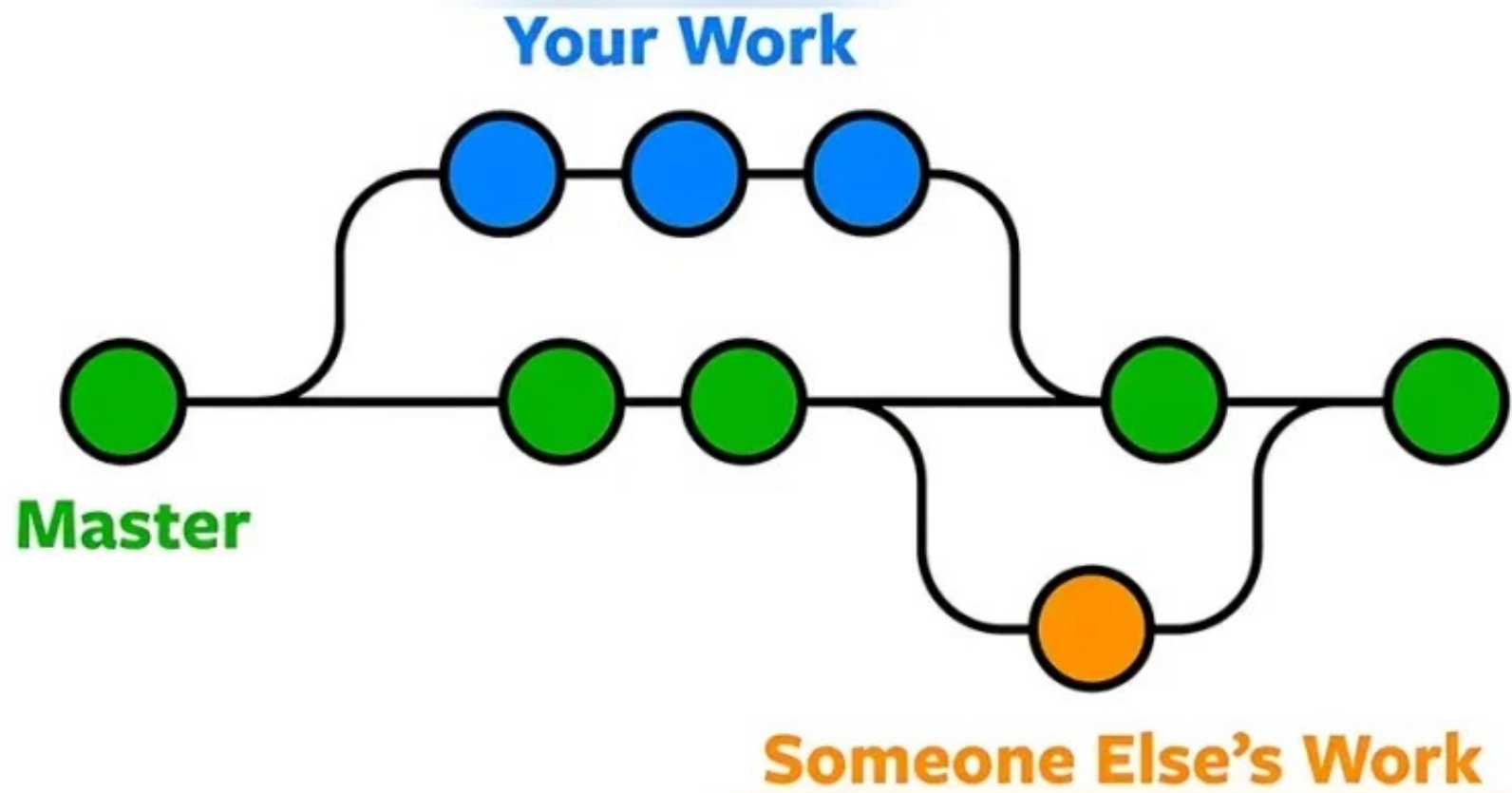
A Solução: Criar ramificações (Branches).



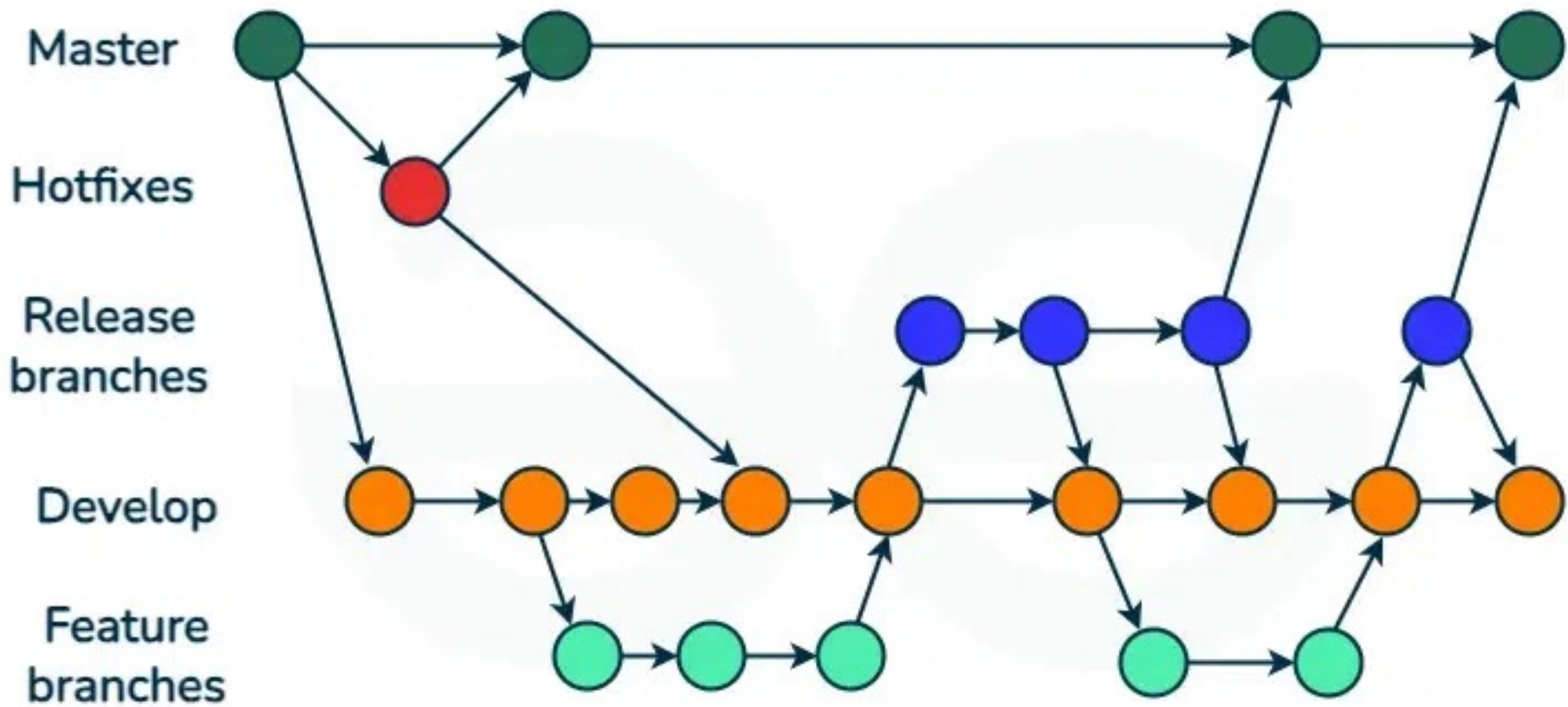
○ Fim da Linha Única

- Imagine que a main é a nossa linha do tempo oficial (o sistema que está no ar). Uma **Branch** é um Universo Paralelo.
- Você clona o universo exato onde está agora, vai para esse universo paralelo, testa, quebra, explode e desenvolve lá.
- A linha do tempo oficial continua intacta.

○ Fim da Linha Única



GitFlow



GitFlow

- É um dos modelos de ramificação (branching) mais famosos e utilizados no mercado para organizar o fluxo de trabalho de equipes de desenvolvimento.
- Ele pega o conceito do "Multiverso do Código" e cria regras estritas sobre **onde** e **como** cada universo deve existir.
- Vamos entender o que cada linha de tempo representa na vida real de uma fábrica de software:
- **1. Master (ou Main)**
 - É o "Palco Principal" ou o ambiente de Produção. É o código oficial que está rodando no servidor e sendo usado pelos clientes neste exato momento.
 - Guarda apenas versões estáveis e finalizadas.
 - **Regra de ouro:** Ninguém nunca, em hipótese alguma, programa diretamente na Master. Cada "bolinha" (commit) nesta linha representa uma nova versão lançada do sistema (ex: v1.0, v1.1, v2.0).

GitFlow

➤ 2. Develop (Desenvolvimento)

- É a "Espinha Dorsal" do desenvolvimento. É a linha do tempo que contém o código para o *próximo* lançamento do sistema.
- É aqui que todas as novas funcionalidades se encontram e são integradas.
- Quando um desenvolvedor termina seu trabalho em uma sala isolada, ele junta (merge) o código aqui.
- É o ambiente onde a equipe de testes geralmente valida se as coisas novas estão funcionando juntas.

GitFlow

➤ 3. Feature branches (Ramos de Funcionalidade)

- São as "Oficinas de Trabalho" isoladas.
- É onde o trabalho pesado do dia a dia acontece.
- Se o chefe pedir para criar uma "Tela de Login" e um "Carrinho de Compras", a equipe cria duas *feature branches* separadas, ambas nascendo a partir da Develop.
- O desenvolvedor trabalha seguro no seu universo, testa tudo e, quando está perfeito, faz o merge de volta para a Develop.

GitFlow

➤ 4. Release branches (Ramos de Lançamento / Homologação)

- É a "Sala de Polimento".
- Imagine que a Develop acumulou 5 funcionalidades novas e a equipe decide que é hora de lançar a versão 2.0 do sistema.
- Cria-se uma branch de Release.
 - A partir desse momento, **nenhuma funcionalidade nova pode entrar aqui.**
- A equipe usa essa branch apenas para corrigir os últimos bugs menores, atualizar manuais e arrumar o número da versão.
- Quando está impecável, ela é unida à Master (vai para o ar) e também unida de volta à Develop (para garantir que os ajustes finos não se percam).

GitFlow

➤ 5. Hotfixes (Correções de Emergência)

- É a "Ambulância" ou o "Extintor de Incêndio".
- O sistema está no ar (Master) e, de repente, descobrem que o botão de pagamento não está funcionando.
- A empresa está perdendo dinheiro. Você não pode esperar o fluxo normal da Develop ou criar uma Release.
- Você puxa uma branch de Hotfix **diretamente da Master**, conserta o erro imediatamente e faz o merge de volta para a Master.
 - Resolvendo o problema do cliente na hora.
- Por fim, você também joga essa correção na Develop para garantir que o bug não volte nas próximas versões.

O Comando **git checkout**

- O Git Checkout é uma das ferramentas mais versáteis do Git, funcionando basicamente como o seu "meio de transporte" entre diferentes estados do seu projeto.
- Pense nele como uma forma de dizer ao Git: "Ei, quero ver como o código estava neste ponto específico agora".
- **1. Trocar de Branch**
- **2. Restaurar Arquivos (Descartar Mudanças)**
- **3. Explorar Commits Antigos**

Aposentando o ~~checkout~~

- Historicamente, usávamos o `git checkout` para tudo (mudar de branch e restaurar arquivos). Isso confundia muito os iniciantes.
- Nas versões recentes, o Git dividiu isso:
 - **`git switch`** (Só para trocar de ramificação/branch).
 - **`git restore`** (Só para restaurar arquivos).
- Foque no restore! É muito mais semântico e fácil para lembrar.

Aposentando o ~~checkout~~

- **O checkout também cria e muda de branches!**
 - Na verdade, ele fazia praticamente *tudo*.
- Historicamente, nós usávamos o git checkout para todas as operações de "mover o ponteiro", o que confundia muito os iniciantes. O comando era como um "canivete suíço" superlotado.
- Veja como era o cenário confuso das versões mais antigas do Git:
- **Para restaurar um arquivo apagado (O nosso contexto de Pânico):**
 - `git checkout main.c`
- **Para viajar para outra branch:**
 - `git checkout nome-da-branch`
- **Para CRIAR e viajar para uma nova branch:**
 - `git checkout -b nova-branch` (onde o -b significava *branch*).

Criando e Viajando entre Universos

- Relembre que combinamos de aposentar o checkout. O comando moderno para viajar é o **switch**.
- **Criar um universo:**
 - `git branch nome-da-ramificacao`
- **Viajar para ele:**
 - `git switch nome-da-ramificacao`
- **O Atalho - Criar e viajar ao mesmo tempo:**
 - `git switch -c nome-da-ramificacao`
 - (O -c significa "create").

O Fechamento do Portal (Merge)

- A funcionalidade ficou pronta no universo paralelo e o código está lindo.
- Como trazer isso para o oficial?
- Você NUNCA puxa de onde você está. Você vai para o destino e "suga" a outra linha do tempo.
- **Passo 1:** Viaje de volta para a base:
 - **`git switch main.`**
- **Passo 2:** Sugue a alteração:
 - **`git merge nome-da-ramificacao.`**

Apagando -d

- Ao usar o -d (minúsculo), o Git verifica se todas as alterações daquela branch já foram mescladas na main.
- Se houver código lá que não foi integrado, o Git **bloqueia** a exclusão e avisa que você está prestes a perder trabalho.
- Este é o comando de segurança
- **git branch -d <nome-da-branch>**

Apagando --delete

- Se já tiver feito o `git push -u origin <nome-da-branch>`, o branch passa a existir no GitHub também.
- Deletá-la no computador local **não** a apaga do site.
- Para limpar a nuvem pelo terminal, usa-se:
- **`git push origin --delete <nome-da-branch>`**
- Esse comando se comunica com o servidor (origin) e pede a exclusão do universo paralelo lá na nuvem.

Apagando --delete

- Se você mandar apagar o universo paralelo lá na nuvem, o GitHub simplesmente corta a etiqueta da branch na mesma hora, **mesmo que você nunca tenha feito o merge** do código para a main.
- **No GitHub:** Se você não tinha feito o merge, os seus commits ficam "órfãos" na nuvem. O *Pull Request* (se existir) é fechado automaticamente e as alterações somem da interface do site. O lixeiro virtual do GitHub vai limpar esses arquivos permanentemente depois de um tempo.
- **No Computador (Codespaces):** Deletar a branch na nuvem **não deleta a branch no seu computador local**. O seu Codespaces continua com a branch intacta.
 - Se você se arrepender, basta dar um `git push -u origin <branch>` novamente e a branch ressuscita na nuvem com todo o código dentro.

Lembre-se!

Não é possível serrar o galho em que se está sentado.

Se eles tentam deletar a branch enquanto estiverem dentro dela, o Git dará erro.

